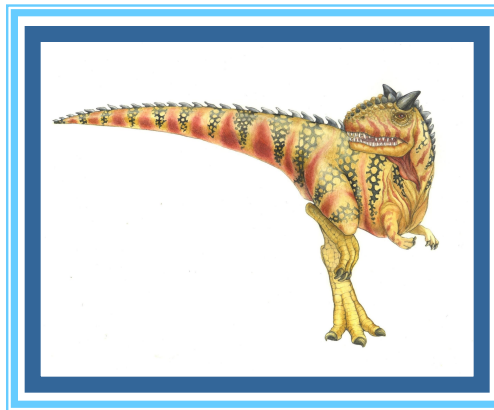


Chapter 1: Introduction





Chapter 1: Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Kernel Data Structures
- Computing Environments
- Open-Source Operating Systems





Objectives

- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems





What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner





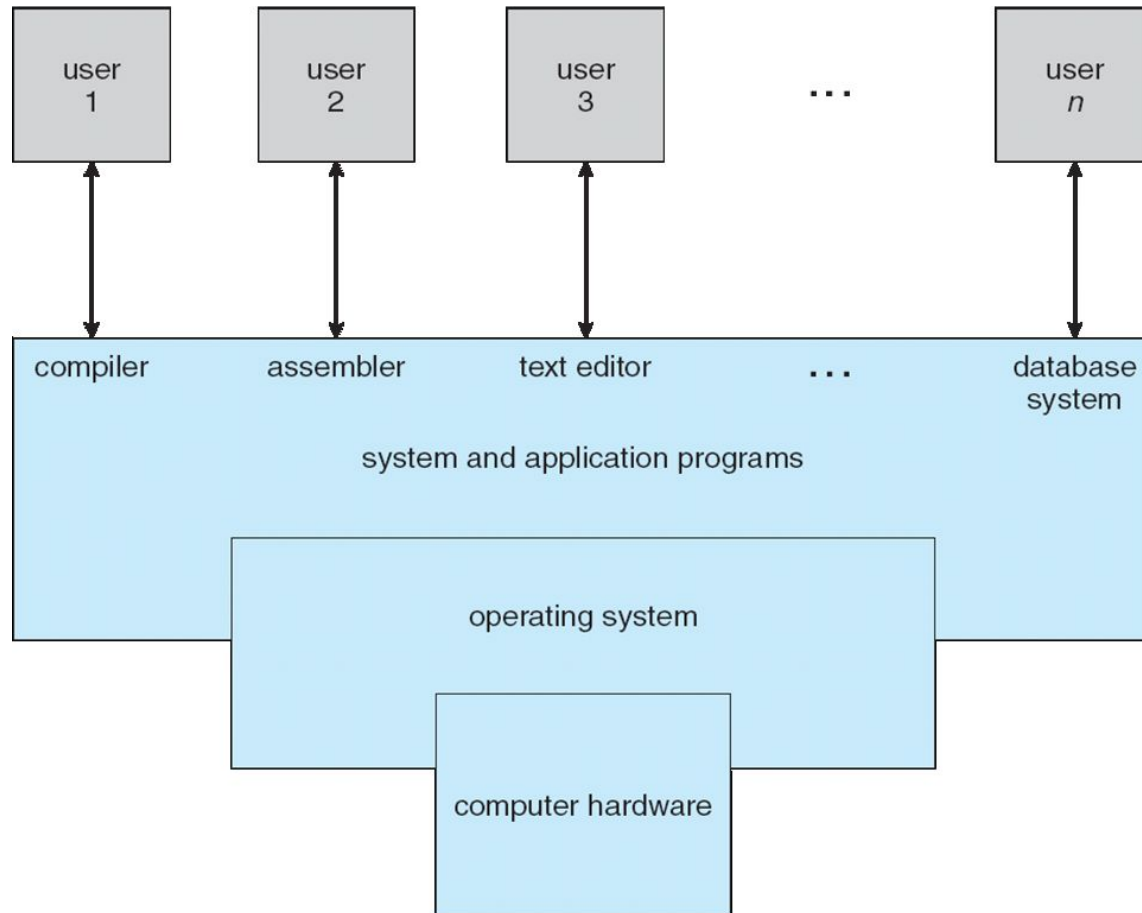
Computer System Structure

- Computer system can be divided into four components:
 - Hardware – provides basic computing resources
 - 4 CPU, memory, I/O devices
 - Operating system
 - 4 Controls and coordinates use of hardware among various applications and users
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - 4 Word processors, compilers, web browsers, database systems, video games
 - Users
 - 4 People, machines, other computers





Four Components of a Computer System





What Operating Systems Do

- Depends on the point of view : User View & System View

User View

- Users want convenience, **ease of use** and **good performance**
 - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- Users of dedicated systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Handheld computers are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles





Operating System Definition

System View:

- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer
 - Emphasizes the need to control the various I/O devices and user programs





Operating System Definition (Cont.)

- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is a good approximation
 - But varies wildly
- “The one program running at all times on the computer” is the **kernel**.
- Everything else is either
 - a system program (ships with the operating system) , or
 - an application program.
- Mobile operating systems often include not only a core kernel but also **middleware**—a set of software frameworks that provide additional services to application developers.





Computer Startup

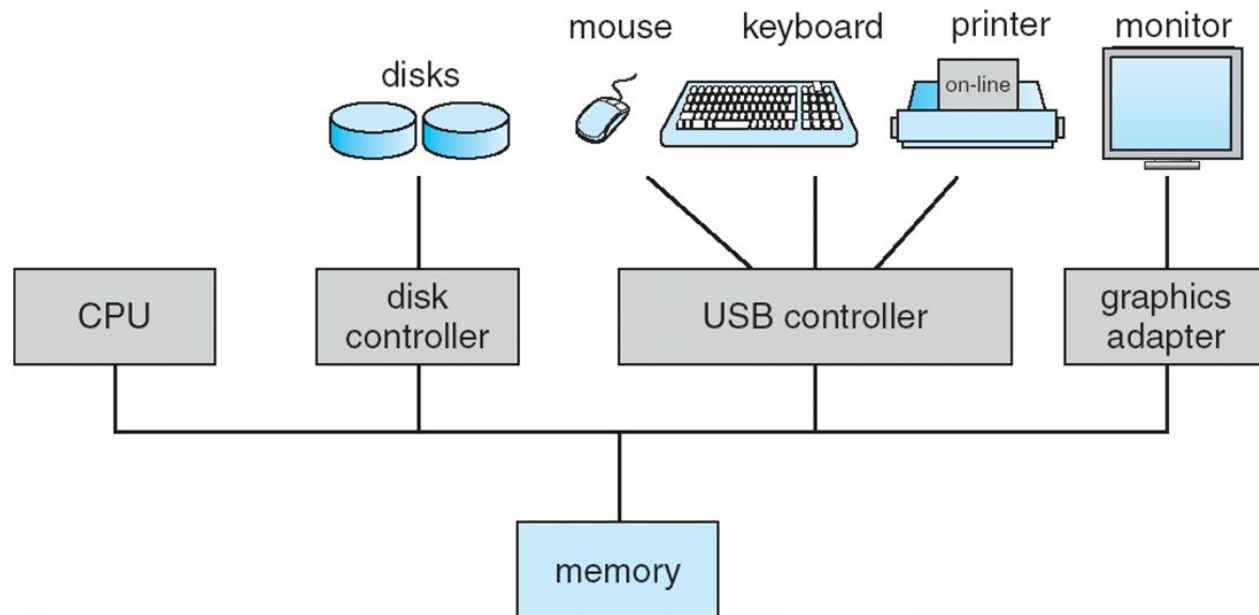
- **bootstrap program** is loaded at power-up or reboot
 - Typically stored in ROM or EPROM, generally known as **firmware**
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution
 - The bootstrap program is loaded into memory from the disk by the system's ROM during the first stage of the loading process.
 - The bootstrap program initializes the CPU, installs banks, and launches the debug agent.
 - It also discovers existing RAM and builds the initial kernel virtual address space and device tree.
 - The kernel is loaded by the bootstrap program and performs memory tests to determine how much RAM is available.
 - The kernel then identifies and configures devices, initializes the system, and starts system processes





1.2 Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles





Computer System Organization

- Computer-system operation
 - Once the kernel is loaded and executing, it can start providing services to the system and its users.
 - Some services are provided outside of the kernel, by system programs that are loaded into memory at boot time to become **system processes**, or **system daemons** that run the entire time the kernel is running.
 - On UNIX, the first system process is “init,” and it starts many other daemons.
 - Once this phase is complete, the system is fully booted, and the system waits for some event to occur.





1.2.1 Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**
- The occurrence of an event is usually signaled by an interrupt from either the hardware or the software.
- **Hardware** may trigger an interrupt at any time by sending a **signal to the CPU**, usually by way of the system bus.
- **Software** may trigger an interrupt by executing a special operation called a **system call (also called a monitor call)**.





1.2.1 Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**





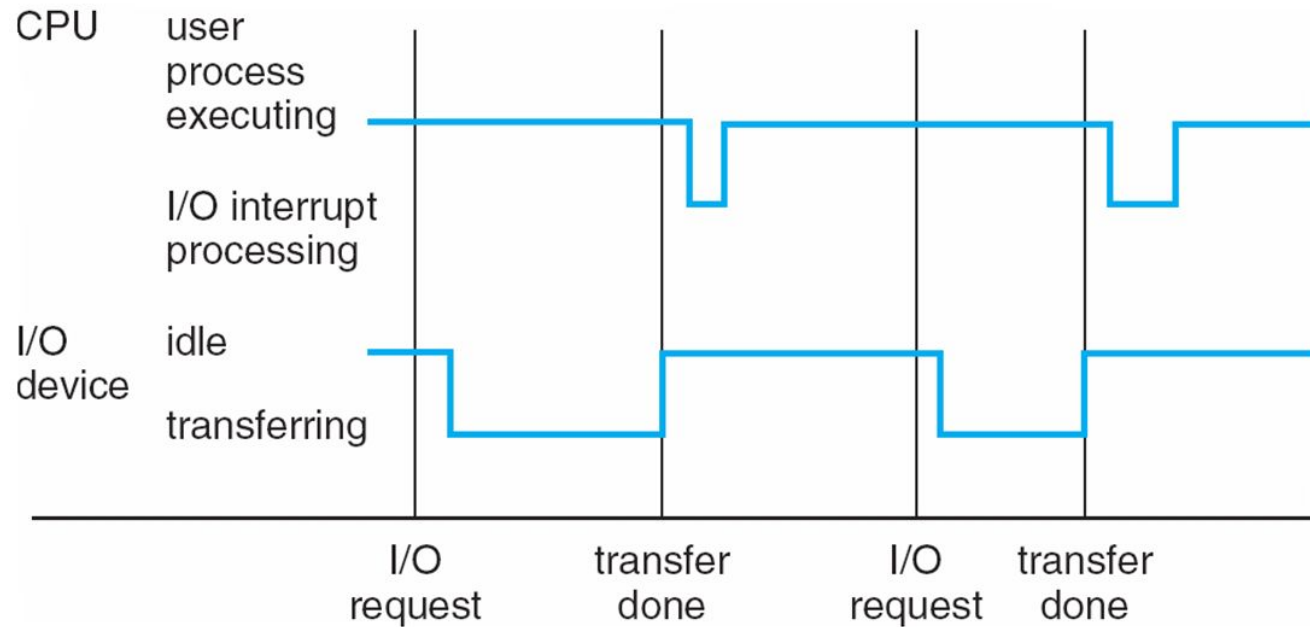
1.2.2 Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - **polling**
 - **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt





Interrupt Timeline





Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is 1,024 bytes

a **megabyte**, or **MB**, is $1,024^2$ bytes

a **gigabyte**, or **GB**, is $1,024^3$ bytes

a **terabyte**, or **TB**, is $1,024^4$ bytes

a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).





1.2.3 Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular
- Another form of nonvolatile storage is **NVRAM**, which is DRAM with battery backup power.
 - This memory can be as fast as DRAM and (as long as the battery lasts) is nonvolatile.





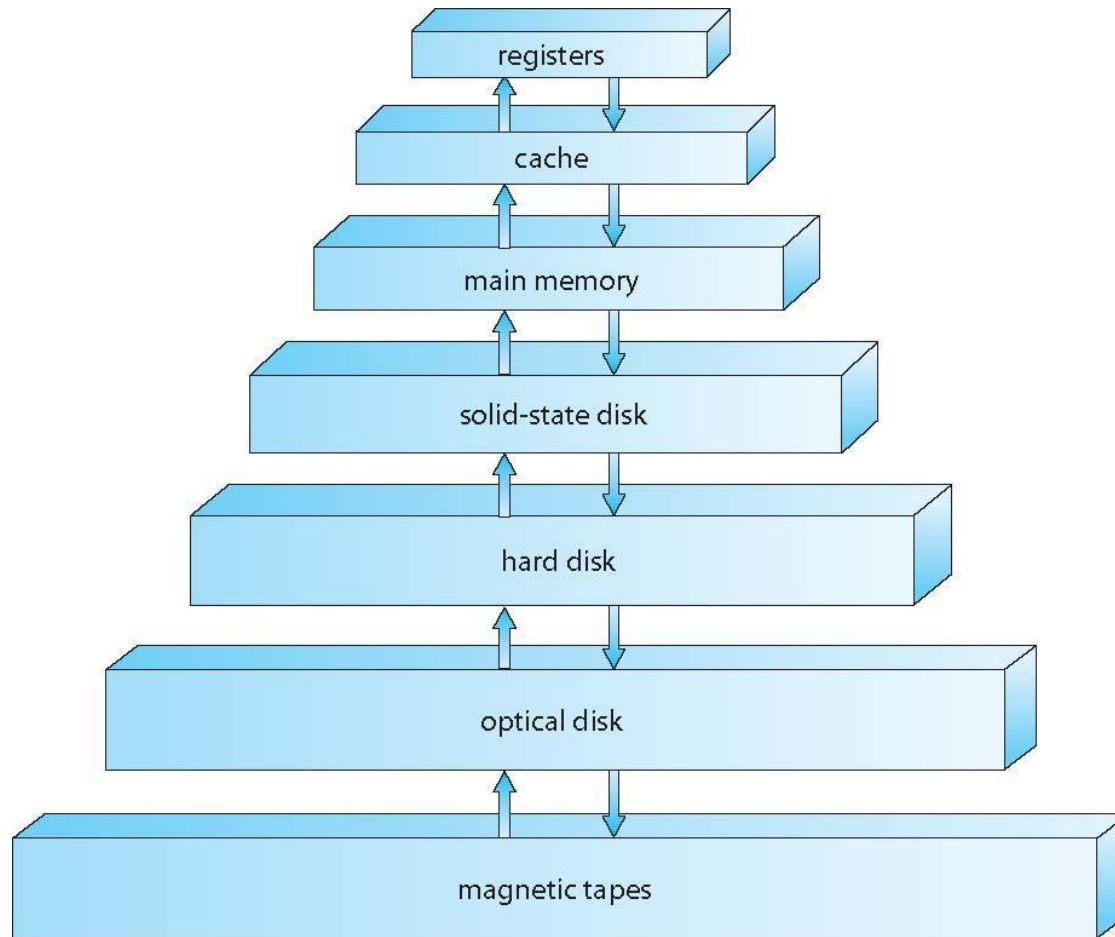
1.2.3 Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel
- Ideally, we want the programs and data to reside in main memory permanently. This arrangement usually is not possible for the following two
- reasons:
 - 1. Main memory is usually too small to store all needed programs and data permanently.
 - 2. Main memory is a volatile storage device that loses its contents when power is turned off or otherwise lost.





Storage-Device Hierarchy





Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy





1.2.4 I/O Structure

- A general-purpose computer system consists of CPUs and multiple device controllers that are connected through a common bus.
- Each device controller is in charge of a specific type of device.
- Depending on the controller, more than one device may be attached.
 - For instance, seven or more devices can be attached to the small computer-systems interface (SCSI) controller.
- A device controller maintains some local buffer storage and a set of special-purpose registers.
- The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.
- Typically, operating systems have a **device driver** for each device controller.
- This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.





I/O Structure

- To **s t a r t** an I/O operation, the device driver loads the appropriate registers within the device controller.
- The device controller, in turn, examines the contents of these registers to determine what action to take (such as “read a character from the keyboard”).
- The controller starts the transfer of data from the device to its local buffer.
- Once the transfer of data is complete, the device **controller informs the device driver via an interrupt** that it has finished its operation.
- The device driver then returns control to the operating system, possibly returning the data or a pointer to the data if the operation was a read.
- For other operations, the device driver returns status information.
- This form of **interrupt-driven I/O** is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as disk I/O.





I/O Structure

- This form of interrupt-driven I/O is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as disk I/O.
- To solve this problem, direct memory access (DMA) is used.
- After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from its own buffer storage to memory, with no intervention by the CPU.
- Only one interrupt is generated per block, to tell the device driver that the operation has completed, rather than the one interrupt per byte generated for low-speed devices.
- While the device controller is performing these operations, the CPU is available to accomplish other works





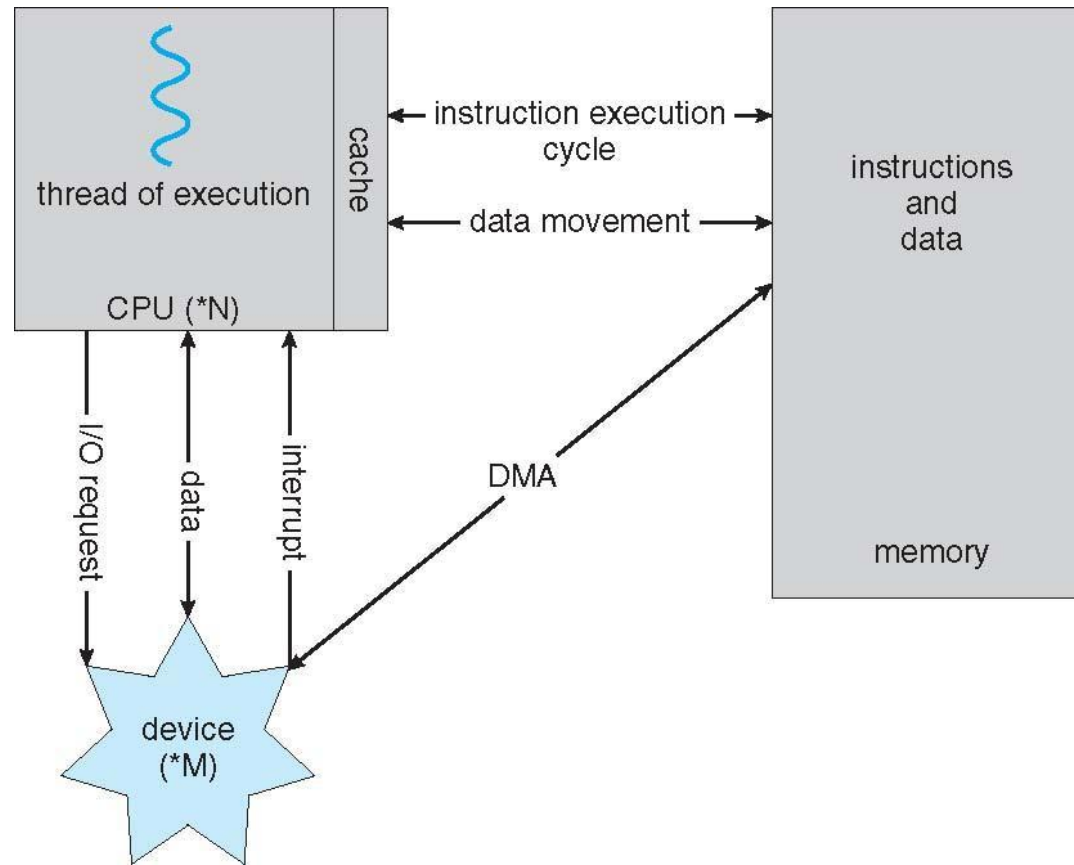
Direct Memory Access Structure

- After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from its own buffer storage to memory, with no intervention by the CPU.
- Only one interrupt is generated per block, to tell the device driver that the operation has completed, rather than the one interrupt per byte generated for low-speed devices.
- While the device controller is performing these operations, the CPU is available to accomplish other work.
- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte





How a Modern Computer Works



A von Neumann architecture





1. 3 Computer-System Architecture

- Most systems use a **single general-purpose processor**
 - Most systems have **special-purpose processors** as well
 - They may come in the form of device-specific processors, such as disk, keyboard, and graphics controllers; or, on mainframes, they may come in the form of more general-purpose processors, such as I/O processors that move data rapidly among the components of the system.
 - All of these special-purpose processors run a limited instruction set and do not run user processes.
 - Sometimes, they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.
 - 4 For example, a disk-controller microprocessor receives a sequence of requests from the main CPU and implements its own disk queue and scheduling algorithm.
 - This arrangement relieves the main CPU of the overhead of disk scheduling.
 - PCs contain a microprocessor in the keyboard to convert the keystrokes into codes to be sent to the CPU





Computer-System Architecture

- **Multiprocessors** systems growing in use and importance (also known as parallel systems or multicore systems)
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include:
 1. **Increased throughput**
 - 4 By increasing the number of processors, we expect to get more work done in less time.
 - 4 The speed-up ratio with N processors is not N , however; rather, it is less than N .
 - 4 When multiple processors cooperate on a task, a certain amount of overhead is incurred in keeping all the parts working correctly.
 - 4 This overhead, plus contention for shared resources, lowers the expected gain from additional processors.
 - 4 Similarly, N programmers working closely together do not produce N times the amount of work a single programmer would produce.





Computer-System Architecture

- **Multiprocessors**
- Advantages include:

2. Economy of scale:

- 4 Multiprocessor systems can cost less than equivalent multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.
- 4 If several programs operate on the same set of data, it is cheaper to store those data on one disk and to have all the processors share them than to have many computers with local disks and many copies of the data.

3. Increased reliability – graceful degradation or fault tolerance

- 4 If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down.
- 4 If we have ten processors and one fails, then each of the remaining nine processors can pick up a share of the work of the failed processor. Thus, the entire system runs only 10 percent slower, rather than failing altogether.





Computer-System Architecture

- **Multiprocessors**

- Two types:

1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.

- A boss processor controls the system; the other processors either
- look to the boss for instruction or have predefined tasks. This scheme defines a boss–worker relationship.
- The boss processor schedules and allocates work to the worker processors.

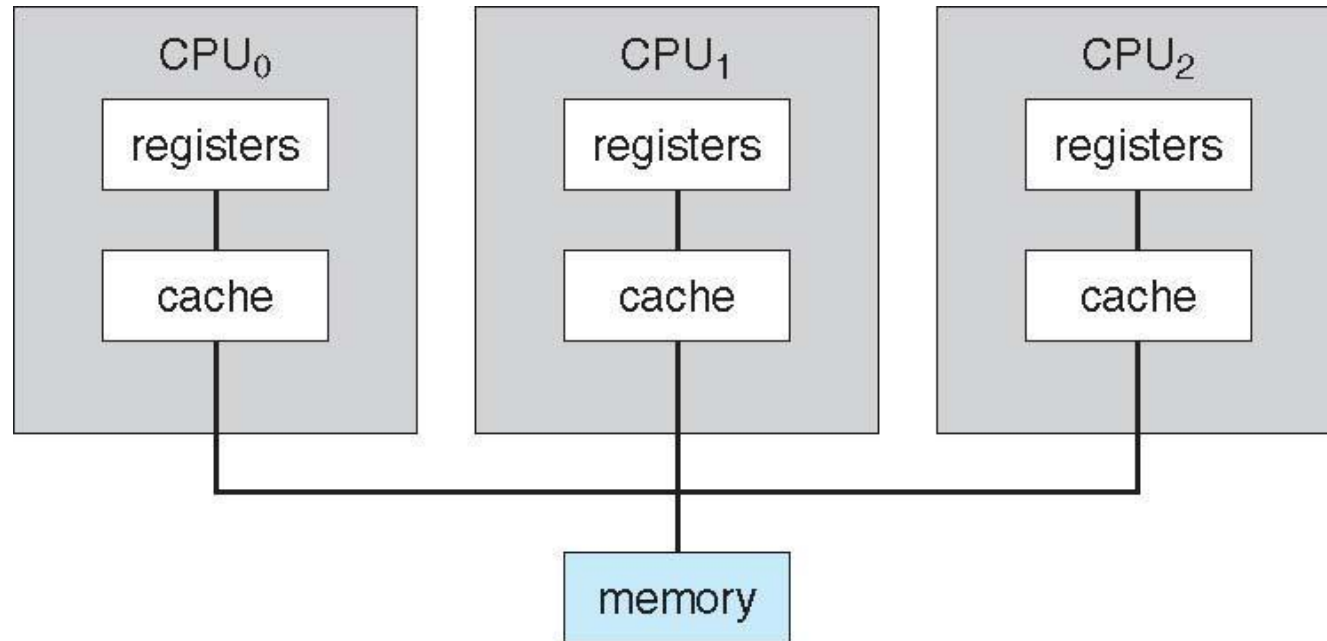
2. **Symmetric Multiprocessing [SMP]**– each processor performs all tasks

- SMP means that all processors are peers; no boss–worker relationship exists between processors.
- Each processor has its own set of registers, as well as a private—or local —cache.
- However, all processors share physical memory. An example of an
- SMP system is AIX, a commercial version of UNIX designed by IBM





Symmetric Multiprocessing Architecture



- Multiprocessing adds CPUs to increase computing power.
- If the CPU has an integrated memory controller, then adding CPUs can also increase the amount of memory addressable in the system.
- Either way, multiprocessing can cause a system to change its memory access model from uniform memory access (UMA) to non-uniform memory access (NUMA).





A Dual-Core Design

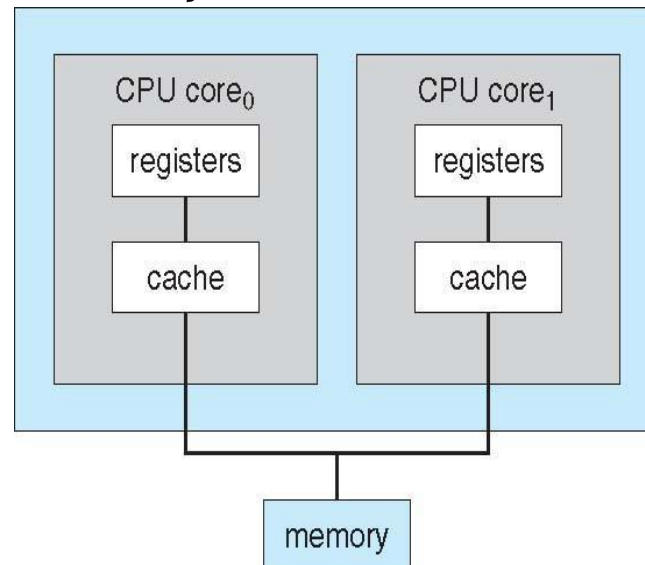
- A recent trend in CPU design is to include multiple computing cores on a single chip.
- Such multiprocessor systems are termed **multicore**.
 - They can be more efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication.
- In addition, one chip with multiple cores uses significantly less power than multiple single-core chips.
- Each core has its own register set as well as its own local cache.
- Other designs might use a shared cache or a combination of local and shared caches.
- Aside from architectural considerations, such as cache, memory, and bus contention, these multicore CPUs appear to the operating system as N standard processors.
- This characteristic puts pressure on operating system designers—and application programmers—to make use of those processing cores.





A Dual-Core Design

- Finally, blade servers are a relatively recent development in which multiple processor boards, I/O boards, and networking boards are placed in the same chassis.
- The difference between these and traditional multiprocessor systems is that each blade-processor board boots independently and runs its own operating system.
- Some blade-server boards are multiprocessor as well, which blurs the lines between types of computers. In essence, these servers consist of multiple independent multiprocessor systems.





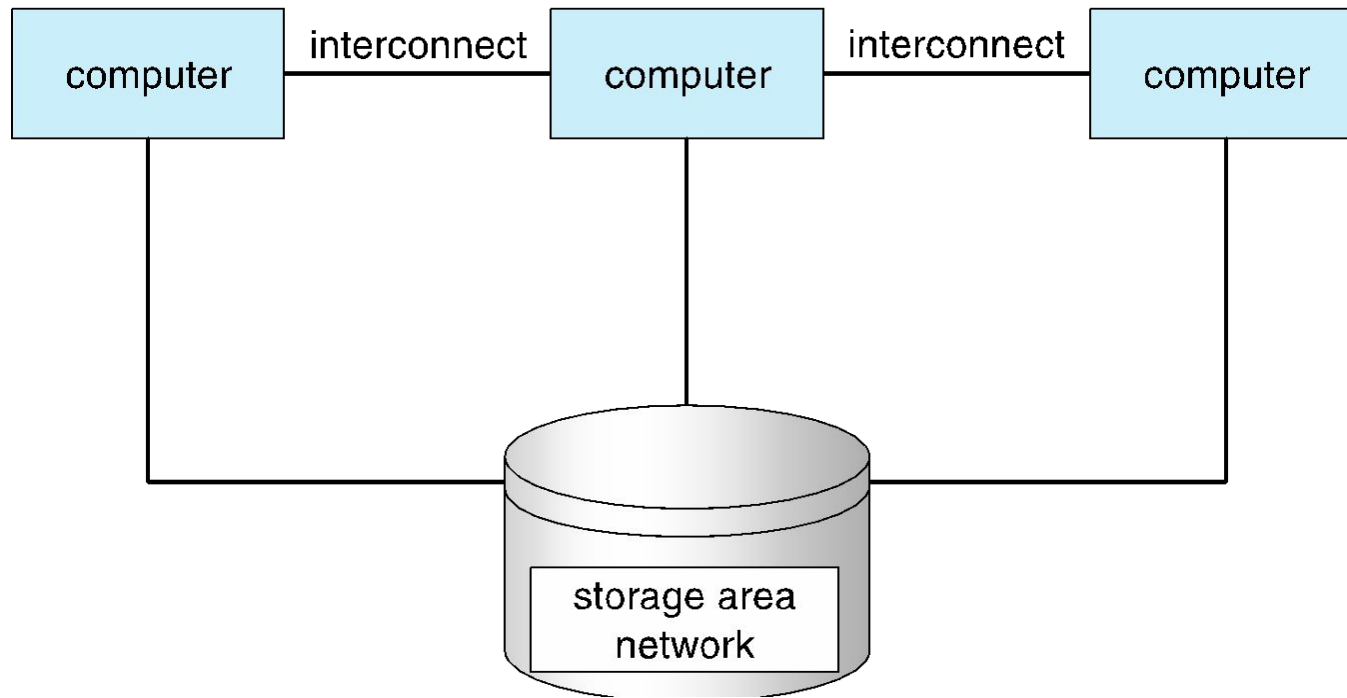
Clustered Systems

- Like multiprocessor systems, but multiple systems working together
 - Usually sharing storage via a **storage-area network (SAN)**
 - **Loosely coupled : Each node may be a single processor system or a multicore system.**
 - Provides a **high-availability** service which survives failures
 - 4 **Asymmetric clustering** has one machine in hot-standby mode
 - The hot-standby host machine does nothing but monitor the active server.
 - If that server fails, the hot-standby host becomes the active server.
 - 4 **Symmetric clustering** has multiple nodes running applications, monitoring each other
 - Some clusters are for **high-performance computing (HPC)**
 - 4 Applications must be written to use **parallelization**
 - Some have **distributed lock manager (DLM)** to avoid conflicting operations





Clustered Systems



- Other forms of clusters:
 - Parallel clusters allow multiple hosts to access the same data on shared storage





1.4 Operating System Structure

- **Multiprogramming (Batch system)** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program executing in memory □ **process**
 - If several jobs ready to run at the same time □ **CPU scheduling**
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory





1.4 Operating System Structure

- **Multiprogramming (Batch system)** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job
 - Since, in general, main memory is too small to accommodate all jobs, the jobs are kept initially on the disk in the job pool.
 - This pool consists of all processes residing on disk awaiting allocation of main memory
 - The set of jobs in memory can be a subset of the jobs kept in the job pool.
 - The operating system picks and begins to execute one of the jobs in memory.
 - Eventually, the job may have to wait for some task, such as an I/O operation, to complete.
 - In a non-multiprogrammed system, the CPU would sit idle.
 - In a multiprogrammed system, the operating system simply switches to, and executes, another job. When that job needs to wait, the CPU switches to another job, and so on.





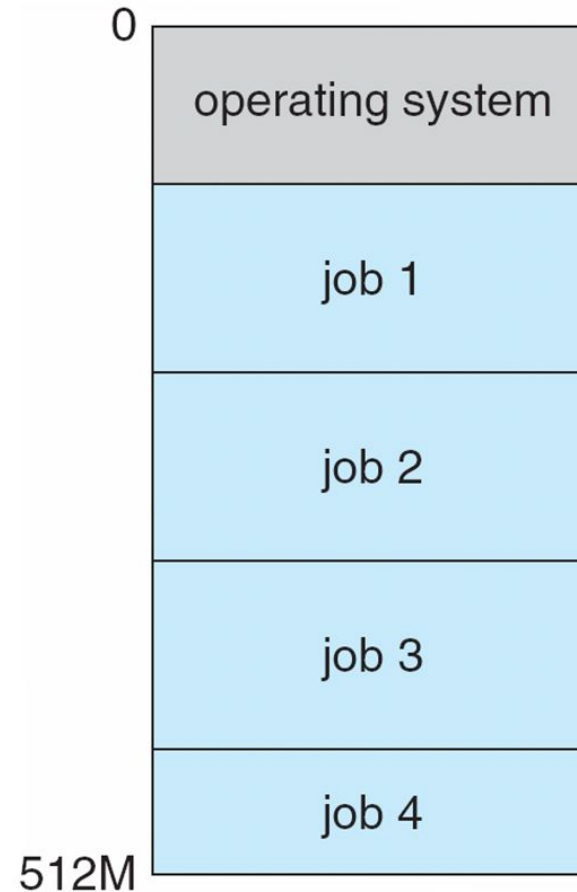
1.4 Operating System Structure

- Time sharing and multiprogramming require that several jobs be kept simultaneously in memory.
- If several jobs are ready to be brought into memory, and if there is not enough room for all of them, then the system must choose among them - Making this decision involves **job scheduling**, When the operating system selects a job from the job pool, it loads that job into memory for execution.
- Having several programs in memory at the same time requires some form of memory management.
- In addition, if several jobs are ready to run at the same time, the system must choose which job will run first. Making this decision is **CPU scheduling**





Memory Layout for Multiprogrammed System





Operating-System Operations

- If there are no processes to execute, no I/O devices to service, and no users to whom to respond, an operating system will sit quietly, waiting for something to happen.
- Events are almost always signaled by the occurrence of an **interrupt or a trap**.
- A trap (or an exception) is a software-generated interrupt caused either by an error (for example, division by zero or invalid memory access) or by a specific request from a user program that an operating-system service be performed.
- The interrupt-driven nature of an operating system defines that system's general structure.
- For each type of interrupt, separate segments of code in the operating system determine what action should be taken.
- Since the operating system and the users share the hardware and software resources of the computer system, we need to make sure that an error in a user program could cause problems only for the one program running.





Operating-System Operations (cont.)

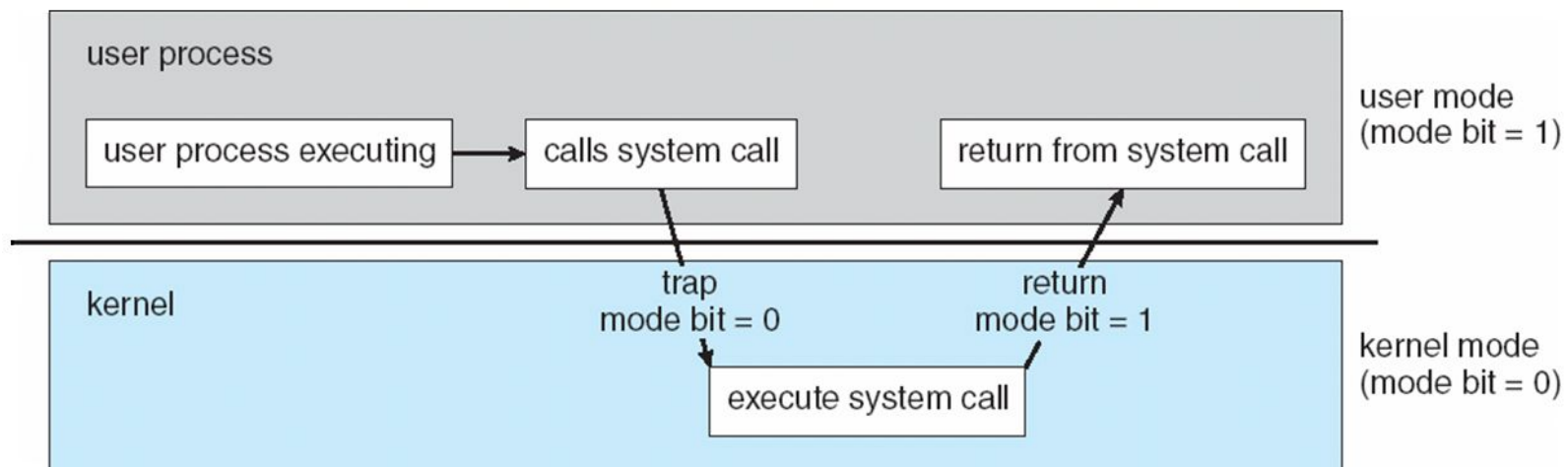
- In order to ensure the proper execution of the operating system, we must be able to distinguish between the execution of operating-system code and user-defined code.
- The approach taken by most computer systems is to provide hardware support that allows us to differentiate among various modes of execution.
- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode** - also called **supervisor mode, system mode, or privileged mode**
 - **Mode bit** provided by hardware
 - 4 Provides ability to distinguish when system is running user code or kernel code
 - 4 Some instructions designated as **privileged**, only executable in kernel mode
 - 4 System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**





Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock.
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a **passive entity**, process is an **active entity**.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads





Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed





Storage Management

- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - **file**
 - Each medium is controlled by device (i.e., disk drive, tape drive)
 - 4 Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - 4 Creating and deleting files and directories
 - 4 Primitives to manipulate files and directories
 - 4 Mapping files onto secondary storage
 - 4 Backup files onto stable (non-volatile) storage media





Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed – by OS or applications
 - Varies between WORM (write-once, read-many-times) and RW (read-write)





Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

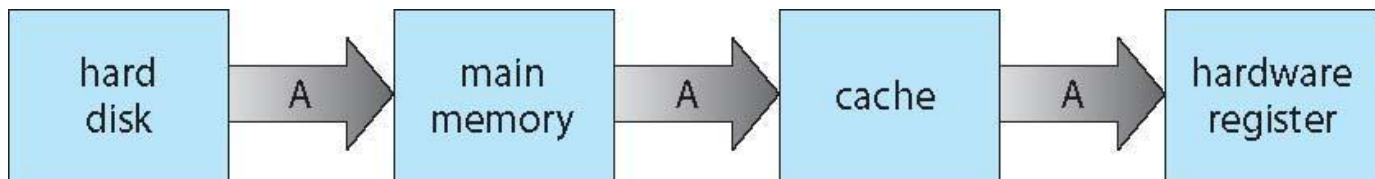
Movement between levels of storage hierarchy can be explicit or implicit





Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist
 - Various solutions covered in Chapter 17





I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices





Protection and Security

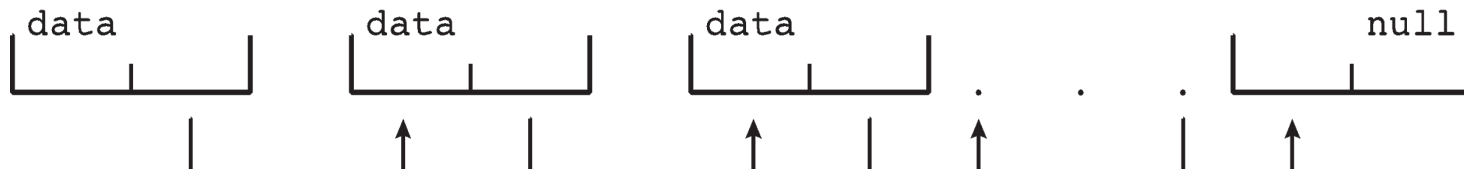
- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights



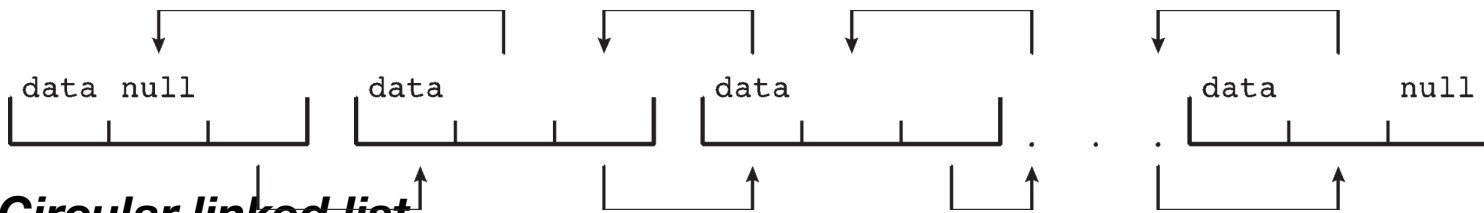


Kernel Data Structures

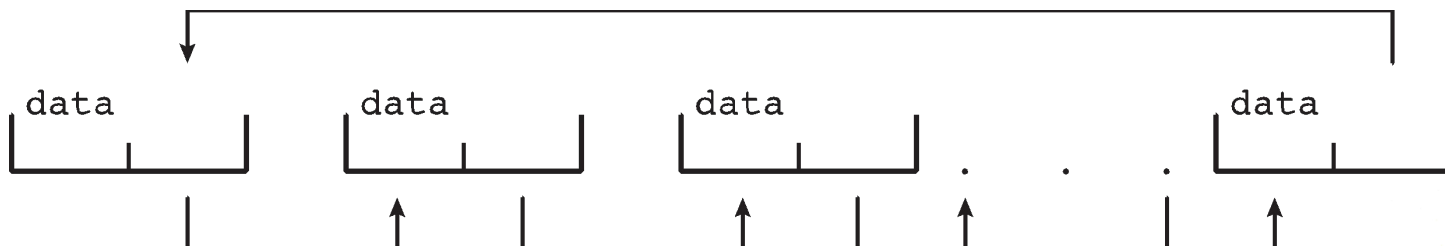
- Many similar to standard programming data structures
- ***Singly linked list***



- ***Doubly linked list***



- ***Circular linked list***



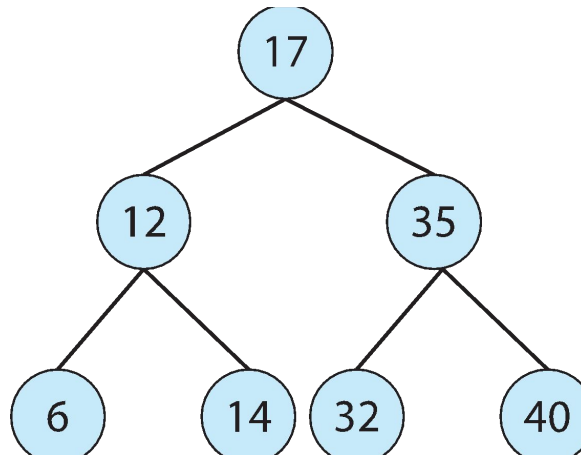


Kernel Data Structures

- **Binary search tree**

left $\leq$ right

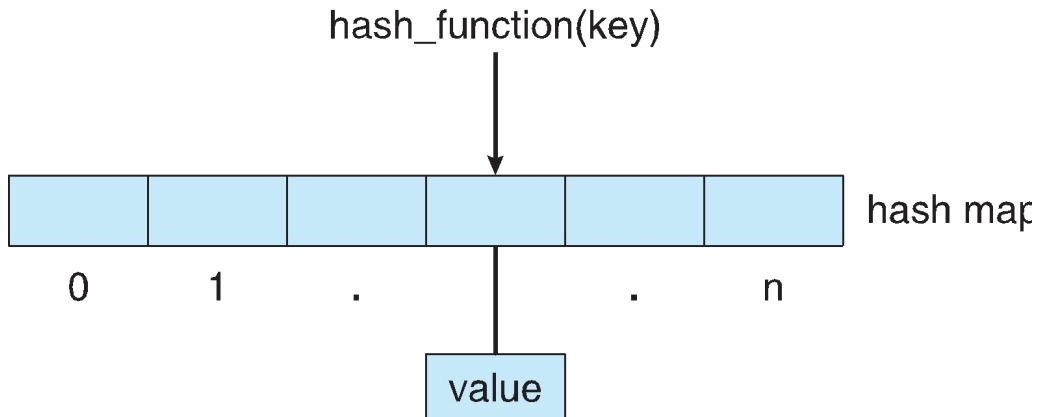
- Search performance is $O(n)$
- **Balanced binary search tree** is $O(\lg n)$





Kernel Data Structures

- **Hash function** can create a **hash map**



- **Bitmap** – string of n binary digits representing the status of n items
- Linux data structures defined in
include files `<linux/list.h>`, `<linux/kfifo.h>`,
`<linux/rbtree.h>`



End of Chapter 1

